SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I **Kruthika Sre S / 192525360** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Kruthika Sre

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	**Automobile Industry**	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	**Healthcare**	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	**Banking**	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	**Cybersecurity**	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	**Manufacturing**	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	**Agriculture**	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	**Telecommunications**	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	**E-Commerce**	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.

9	**Power & Energy**	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	**Human Resources**	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

6. CROP-DISEASE ADVISORY EXPERT SYSTEM

Abstract

The Crop-Disease Advisory Expert System is a rule-based system designed to assist farmers in identifying possible crop diseases from observable symptoms and environmental conditions. The system considers factors such as crop type, soil condition, weather conditions, and visible symptoms including yellow leaves, spots, wilting, poor growth, and pest infestation. Using production rules and logical inference in Prolog, the system identifies possible diseases and recommends suitable actions. The system demonstrates knowledge representation, rule matching, and automated reasoning using forward and backward chaining.

Introduction:

Crop diseases can significantly affect agricultural productivity and crop quality. Farmers often identify diseases by observing symptoms on plants, but accurate identification may require specialized knowledge. An expert system can provide preliminary disease identification by combining crop information, environmental conditions, and visible symptoms. The proposed system represents agricultural knowledge as facts and production rules in Prolog and generates disease-related recommendations through logical inference.

Problem Statement

Farmers may face difficulties in identifying crop diseases at an early stage based only on visible symptoms. Similar symptoms can occur under different diseases or environmental conditions. Therefore, an expert system is developed to analyze crop, soil, weather, and symptom information and identify possible diseases while providing suitable advisory actions.

Objectives

Objective	Description
Crop Identification	Identify the crop being analyzed.
Symptom Analysis	Analyze symptoms such as yellow leaves, spots, wilting, and poor growth.
Environmental Analysis	Consider soil and weather conditions affecting disease occurrence.
Disease Identification	Determine possible crop diseases using production rules.
Advisory Recommendation	Recommend suitable actions based on the identified disease.

Domain Analysis & Knowledge Acquisition

The domain consists of crop health, crop diseases, environmental conditions, symptoms, and agricultural interventions. Knowledge is acquired from agricultural disease identification guidelines and expert knowledge relating crop symptoms to possible diseases.

The major knowledge categories are:

Category	Examples
Crop	Tomato, Rice, Potato
Soil Condition	Dry soil, wet soil, nutrient-deficient soil
Weather	Humid, rainy, hot, dry
Symptoms	Yellow leaves, leaf spots, wilting, poor growth
Pest Condition	Pest infestation, no infestation
Disease	Leaf blight, bacterial wilt, fungal disease
Action	Remove infected parts, improve drainage, pest control

Knowledge Representation & Production Rules

The system represents agricultural knowledge using facts and production rules. Facts describe the crop and observed conditions, while production rules connect these conditions to possible diseases and recommended actions.

Example facts:

crop(tomato).

soil(tomato, wet).

weather(tomato, humid).

symptom(tomato, yellow_leaves).

symptom(tomato, leaf_spots).

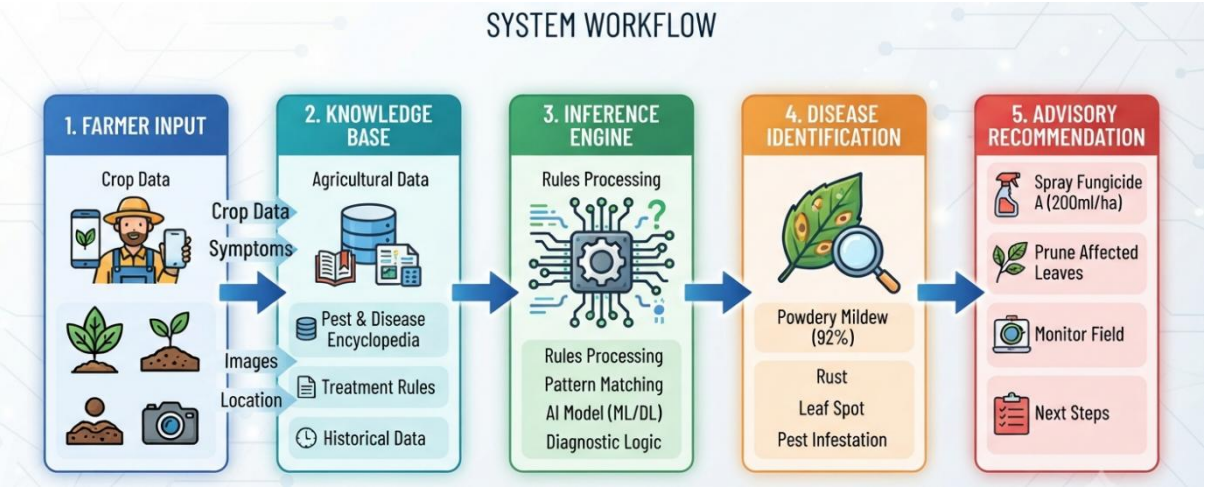
Example production rules:

Rule	Production Rule
R1	IF crop is tomato AND leaves have spots AND weather is humid THEN possible disease is early blight.
R2	IF crop is tomato AND plant is wilting AND soil is wet THEN possible disease is bacterial wilt.
R3	IF crop has yellow leaves AND poor growth THEN possible nutrient deficiency.
R4	IF pest infestation is present AND leaves are damaged THEN recommend pest control.
R5	IF soil is excessively wet AND plant is wilting THEN recommend improving drainage.

Expert System Architecture

The proposed system consists of the following major components:

Component	Function
User Input	Accepts crop, soil, weather, and symptom information.
Knowledge Base	Stores agricultural facts and production rules.
Inference Engine	Matches facts with rules and derives conclusions.
Disease Identification Module	Determines possible crop diseases.
Recommendation Module	Provides suitable actions to the farmer.
Output	Displays the identified disease and advisory recommendation.



System Architecture

Inference Mechanism & Algorithms

The system uses forward chaining and backward chaining for logical reasoning.

Forward Chaining:

Forward chaining starts with known facts such as crop type, soil condition, weather, and symptoms. The inference engine applies matching production rules and derives new conclusions until a disease or recommendation is obtained.

Example:

Facts:

Tomato + humid weather + leaf spots

↓

Rule Matching

↓

Early Blight

↓

Recommended Action

Backward Chaining:

Backward chaining starts with a possible conclusion, such as early blight, and works backward to determine whether the required symptoms and environmental conditions are present.

Goal: Early Blight

↓

Check required conditions

↓

Humid weather + leaf spots

↓

Conditions satisfied

↓

Early Blight confirmed as possible

Prolog Implementation

The system is implemented using SWI-Prolog. Prolog is suitable because its declarative nature allows agricultural knowledge to be represented directly as facts and rules.

A simplified implementation contains:

```
crop(tomato).
soil(tomato, wet).
weather(tomato, humid).
symptom(tomato, yellow_leaves).
symptom(tomato, leaf_spots).
disease(Tomato, early_blight) :-
    crop(Tomato),
```

```
    weather(Tomato, humid),
    symptom(Tomato, leaf_spots).
disease(Tomato, bacterial_wilt) :-
    crop(Tomato),
    soil(Tomato, wet),
    symptom(Tomato, wilting).
action(early_blight, remove_infected_leaves).
action(early_blight, apply_fungicide).
action(bacterial_wilt, improve_soil_drainage).
```

The user can query the system to identify possible diseases and obtain recommended actions.

Test Cases & Results

The system is tested using different combinations of crop, environmental conditions, and symptoms.

Test Case	Input Conditions	Expected Result
TC1	Tomato + humid weather + leaf spots	Early blight
TC2	Tomato + wet soil + wilting	Bacterial wilt
TC3	Crop + yellow leaves + poor growth	Possible nutrient deficiency
TC4	Crop + pest infestation + damaged leaves	Recommend pest control
TC5	Wet soil + wilting	Recommend improving drainage

Result:

The system successfully matches the provided facts with the corresponding production rules and generates the expected disease identification or advisory recommendation.

Analysis, Limitations & Future Enhancements

Analysis:

The expert system provides a systematic approach for preliminary crop-disease identification. Prolog's rule-based reasoning makes the decision process transparent and allows agricultural knowledge to be modified or extended easily.

Limitations:

- The system depends on the accuracy of the symptoms entered by the user.
- Only diseases represented in the knowledge base can be identified.
- It provides advisory suggestions rather than laboratory-level diagnosis.
- Environmental conditions may vary between different agricultural regions.

Future Enhancements:

- Add more crops and diseases to the knowledge base.
- Integrate real-time weather and soil sensors.
- Include image-based crop disease detection.
- Develop a graphical or web-based user interface.
- Add machine-learning techniques for improved disease prediction.

Conclusion

The Crop-Disease Advisory Expert System demonstrates how artificial intelligence and knowledge-based reasoning can assist farmers in preliminary crop-disease identification. By representing agricultural knowledge through facts and production rules, the system analyzes crop conditions and visible symptoms to identify possible diseases and recommend suitable actions. The implementation in Prolog demonstrates declarative programming, logical inference, rule matching, and automated conclusion generation using forward and backward chaining.

References:

1. I. Bratko, Prolog Programming for Artificial Intelligence, Pearson.
2. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, Pearson.
3. SWI-Prolog Documentation, SWI-Prolog Reference Manual.
4. Food and Agriculture Organization of the United Nations (FAO), Plant Production and Protection Resources.

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference traces.	7–8% – Provides relevant test cases with mostly accurate results.	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				